

# Assignment 3: Semantic Analyzer for MiRust

Version: v1.0

## 1. OBJECTIVES

- Understand static semantic analysis, symbol table management, and scope rules.
  - Implement type checking and type inference algorithms for a block-structured language.
  - Detect and report semantic errors such as redeclarations, undeclared identifiers/functions/structs, type mismatches in statements and expressions, loop control errors, and assignments to immutable bindings.
  - Develop a Python unit testing suite to verify the correctness of the semantic checker.
- 

## 2. REQUIRED TASKS TO COMPLETE

- **Task 1: Study the AST Node definitions**
    - Re-read `src/Utils/AST.py` to ensure you understand all the abstract syntax tree representations.
  - **Task 2: Implement the StaticChecker.py file**
    - Complete the class `StaticChecker` in `src/checker/StaticChecker.py`.
    - Walk the AST to perform scope checking, type resolution, type checking, and mutability analysis.
    - Raise the appropriate semantic exceptions upon detecting any violation.
  - **Task 3: Write 100 semantic checker tests**
    - Create 100 test cases in `test/test-checker.py` ( `test_1` to `test_100` ) using the provided test harness.
- 

## 3. SEMANTIC ERROR & EXCEPTION TYPES

The static semantic checker must detect violations of MiRust type and scope rules and raise the corresponding exception classes defined in `src/checker/StaticError.py`.

Error reporting adheres to the following principles:

- **Fail-Fast Model:** The semantic analyzer operates strictly under the **fail-fast** principle. As soon as the analyzer detects the first static semantic violation (e.g., encountering an `Undeclared` identifier or a `TypeMismatch` ), it must immediately `raise` the corresponding exception and abort further traversal. The analyzer must **not** attempt error recovery or synthesize `Unknown` types to continue checking.
- **Single-Error Test Guarantee:** To completely prevent grading disputes and ambiguities regarding AST traversal order across different implementations, **every test case in the assignment test suite is guaranteed to contain at most ONE single semantic error** (or be a valid program with 0 errors).

### 3.1. Detailed Exception Class Descriptions

1. **TypeMismatchInStatement(statement)** :
  - Raised when statement type rules are violated (e.g., conditions of `if / while` not being `bool` , or `switch` expression not being `i32` or `bool` , or return types mismatch function signatures). Note: Checking does not require control-flow reachability analysis for

missing `return` statements; it only validates type correctness when encountering `Return` statements.

- `statement` : the AST node representing the invalid statement.

2. **TypeMismatchInExpression(expression) :**

- Raised when operands mismatch operator requirements, arguments mismatch parameter types.
- `expression` : the AST node representing the invalid expression.

3. **CannotAssignToConstant(expression) :**

- Raised when mutating an immutable variable, function parameter, or immutable loop variable. (Matches *Mutability Violations* in Spec §9.4).
- `expression` : the AST node representing the invalid assignment.

4. **Undeclared(kind, identifier) :**

- Raised when a variable, function, struct, type alias, or field is referenced before being declared. Note: If a custom type signature references an undeclared identifier, it must raise `Undeclared(Type, name)`.
- `kind` : one of `Variable`, `Function`, `Struct`, `Field`, `TypeAlias`, `Type`.

5. **Redeclared(kind, identifier) :**

- Raised when an entity is declared multiple times in the same scope.
- `kind` : one of `Variable`, `Function`, `Struct`, `Field`, `TypeAlias`, `Type`, `Parameter`.

6. **UseAfterMoveOrUninitialized(identifier) :**

- Raised when accessing a variable identifier that is uninitialized or has had its ownership moved away (for `Box<T>` variables). Note: Checking is strictly performed at the whole-variable identifier level ( `Id` ). Struct fields or array elements are not tracked statically.
- `identifier` : the name of the variable.

7. **NoEntryPoint() :**

- Raised when the program lacks a valid `fn main() -> void` function.

8. **TypeResolutionError(decl) :**

- Raised when the compiler cannot infer a concrete type for a declaration (e.g. `let x = nil;` ) or on cyclic type references in struct or alias definitions.
- `decl` : the AST node representing the invalid declaration ( `VarDecl`, `StructDecl`, or `TypeAliasDecl` ). For indirectly recursive cycles (e.g., `A -> B -> A`), this must be the AST node of the last declaration in the physical code order that completes the cycle.

9. **InvalidControlFlow(statement) :**

- Raised when `break` or `continue` is used outside of a valid loop or switch context.
- `statement` : the AST node representing the invalid control flow statement ( `Break`, `Continue` ).

10. **StructInitializationError(struct\_name) :**

- Raised when a struct field is uninitialized or initialized multiple times.
- `struct_name` : the name of the struct containing the uninitialized field or re-initialized field.

11. **GlobalInitializationError(identifier) :**

- Raised when a global variable is uninitialized or uses a non-constant expression.
- `identifier` : the name of the global variable.

### 3.2. Mapping Table: Language Spec §9 to StaticError Exception Classes

The table below maps each error category defined in Section 9 of `language_spec_v1.0.md` to the corresponding exception class in `StaticError.py` :

| Language Spec Section | Error Category                 | Exception Class             |
|-----------------------|--------------------------------|-----------------------------|
| §9.3                  | Type Mismatch (Statement)      | TypeMismatchInStatement     |
| §9.3                  | Type Mismatch (Expression)     | TypeMismatchInExpression    |
| §9.4                  | Mutability Violations          | CannotAssignToConstant      |
| §9.5                  | Undeclaration Errors           | Undeclared                  |
| §9.6                  | Redeclaration Errors           | Redeclared                  |
| §9.7                  | Use-After-Move / Uninitialized | UseAfterMoveOrUninitialized |
| §9.8                  | No Entry Point                 | NoEntryPoint                |
| §9.9                  | Type Resolution Errors         | TypeResolutionError         |
| §9.10                 | Invalid Control Flow           | InvalidControlFlow          |
| §9.11                 | Struct Initialization Errors   | StructInitializationError   |
| §9.12                 | Global Initialization Errors   | GlobalInitializationError   |

## 4. REQUISITES & SETUP

- Execute your unit tests: `python src/run.py test-checker`

## 5. PROJECT STRUCTURE

```
root/
├── src/
│   ├── grammar/
│   │   ├── MiRust.g4      # Student's ANTLR4 grammar file
│   │   └── (generated files)
│   ├── astgen/
│   │   └── ASTGenerator.py # Student's AST generator visitor
│   ├── checker/
│   │   ├── StaticChecker.py # Student's semantic analyzer (Assignment 3)
│   │   └── StaticError.py   # Semantic error definitions
│   └── utils/
│       ├── AST.py          # AST node definitions
│       ├── Visitor.py      # Base visitor class for AST traversal (provided)
│       └── lexerror.py     # Lexer error utilities
├── test/
│   └── test-checker.py     # Student's 100 semantic unit tests
└── requirements.txt
```

## 6. EVALUATION CRITERIA & GRADING

- **Semantic Analysis Correctness (100%):** Grade is calculated based on passing the automated test cases run against your `StaticChecker.py` implementation.
- **Fail-Fast & Single-Error Testing:** Each test case contains at most one static semantic error. Your checker should raise the exception immediately upon encountering that error.

Test Distribution Table

| Category                      | Tests              | Note                                                   |
|-------------------------------|--------------------|--------------------------------------------------------|
| Redeclaration & Undeclaration | test_1 – test_19   | Redeclared, Undeclared entities                        |
| Type Mismatch in Statement    | test_20 – test_38  | TypeMismatchInStatement                                |
| Type Mismatch in Expression   | test_39 – test_60  | TypeMismatchInExpression                               |
| Cannot Assign to Constant     | test_61 – test_69  | CannotAssignToConstant                                 |
| Type Resolution & Global Init | test_70 – test_79  | TypeResolutionError, GlobalInitializationError         |
| Struct Init & Move Semantics  | test_80 – test_90  | StructInitializationError, UseAfterMoveOrUninitialized |
| Control Flow & No Entry Point | test_91 – test_100 | InvalidControlFlow, NoEntryPoint                       |
| <b>Total</b>                  | <b>100</b>         |                                                        |

- **Test Cases Penalty:** Writing at least 100 unit tests in `test/test-checker.py` is mandatory. A penalty of 10% will be applied if the test file is missing or contains fewer than 100 test cases.

---

## 7. DELIVERABLES

Students must submit:

- File `src/checker/StaticChecker.py` containing the semantic checker implementation.
- File `test/test-checker.py` containing exactly 100 semantic analyzer tests.

---

## 8. DEADLINE

- **Submission Deadline:** Refer to the [deadlines.json](#) Gist file for the exact due date for Assignment 3.

---

## 9. CHANGELOG

- First release under v1.0.